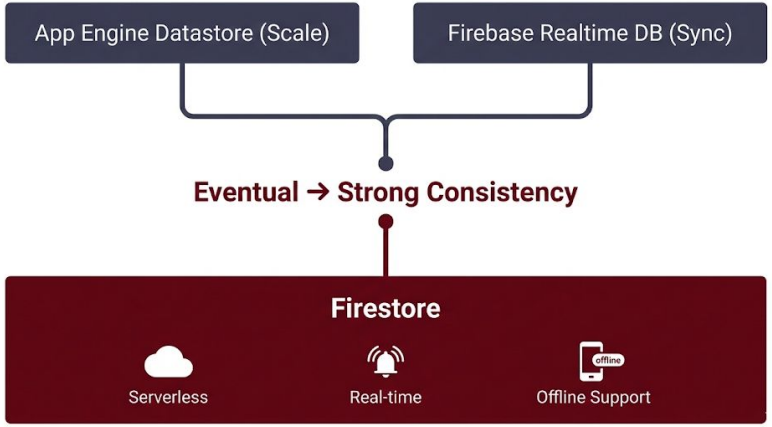
Firestore: The NoSQL Serverless Database for the Application Developer

Source Paper: Kesavan et al. (Google Research / Tsinghua University)

Antonio Pietro Romito - 1932500
Mobile Application & Cloud Computing - A.Y. 2025/2026
Sapienza University of Rome

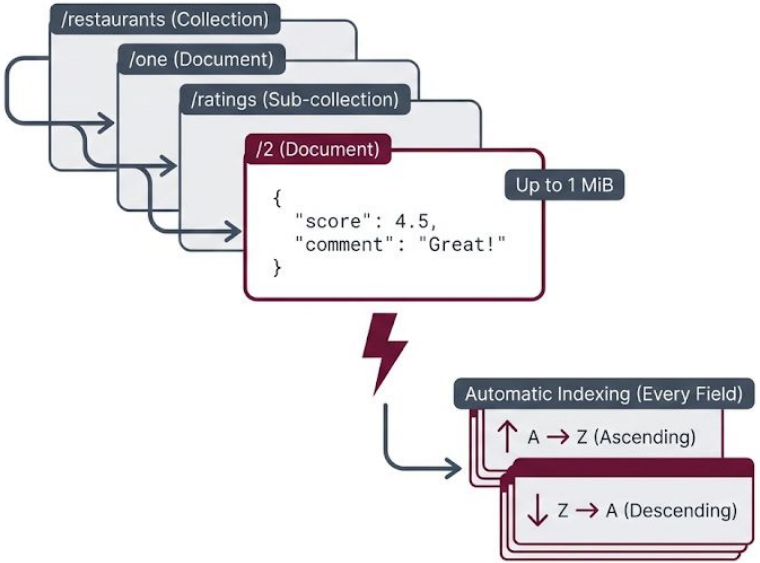
From Datastore to Firestore

- **History:** Evolved from App Engine Datastore (Megastore/Bigtable) and Firebase Realtime Database.
- **Key Limitations Solved:** moved from eventual consistency to **strong consistency**.
- **Core Design Goals:**
  - **Fully Serverless:** No provisioning; scales from 0 bytes to petabytes.
  - **Real-time Queries:** Push notifications to clients on data changes.
  - **Disconnected Operation:** Crucial for mobile reliability.

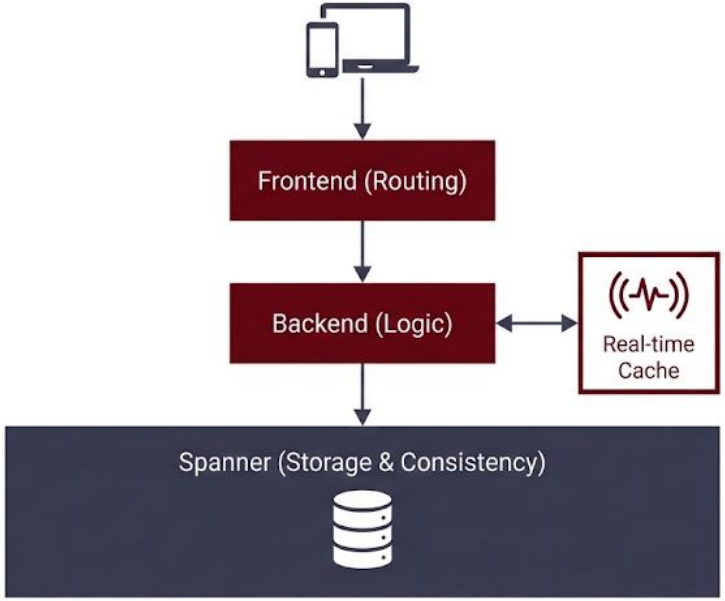


Schemaless Document Store

- **Structure:** Hierarchical model of **Collections** and **Documents.**
- **Documents:** Key-value pairs (up to 1 MiB) supporting primitive and complex types (maps/arrays).
- **Naming:** /restaurants/one/ratings/2 (Collection/Doc/Sub-collection/Doc).
- **Automatic Indexing:**
  - By default, *every* field is indexed (ascending and descending).
  - Enables complex queries without manual database configuration.



The Four Key Components



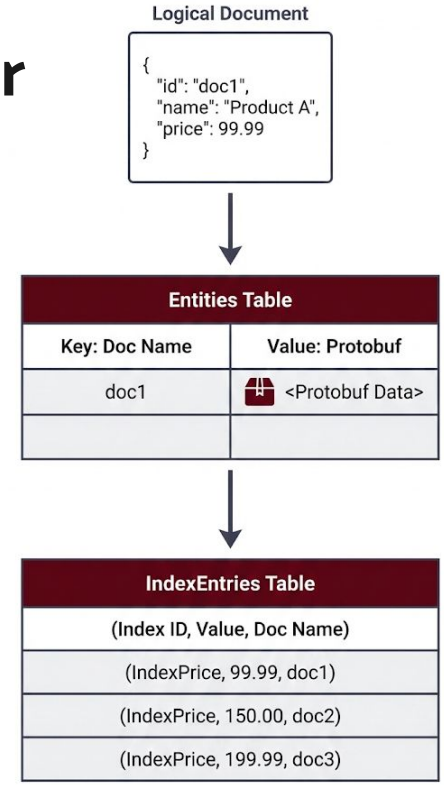
1. **Frontend:** Routes global traffic to the correct region.
2. **Backend:** Translates Firestore requests into Spanner reads/writes.
3. **Real-time Cache:** Manages subscriptions and delta updates for live queries.
4. **Storage Layer (Spanner):** Provides the underlying consistency and replication.

Multi-tenancy: All components are shared across millions of databases to ensure cost-efficiency and isolation.

Mapping NoSQL to Relational Spanner

To enable multi-tenancy, Firestore maps all customer data into two fixed-schema Spanner tables:

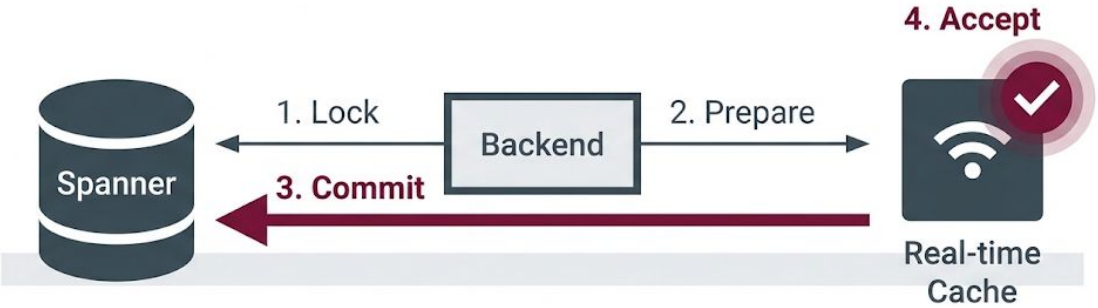
1. **Entities Table:**
   - Stores the actual document data.
   - **Key:** Document Name.
   - **Value:** Protobuf of the document content.
2. **IndexEntries Table:**
   - Stores the inverted indexes.
   - **Key:** (Index ID, Value, Document Name).
   - **Result:** Sorted rows allow efficient linear scans for queries.



Ensuring Consistency (ACID)

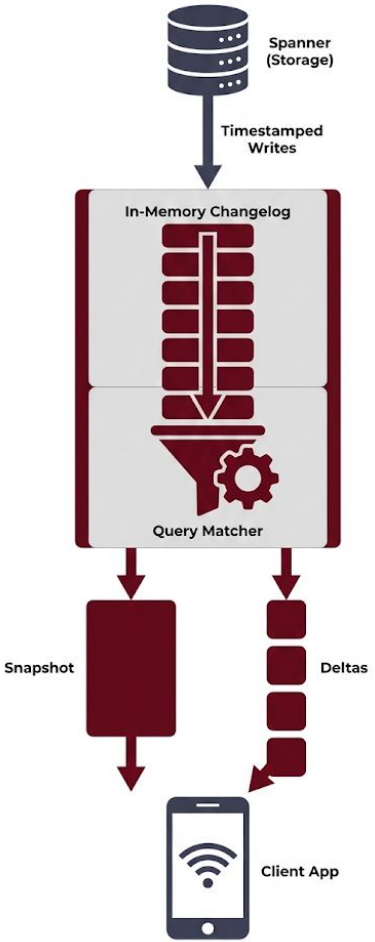
A write involves a **2-Phase Commit** between Spanner and the Real-time Cache:

1. **Lock:** Backend acquires locks on the Document and Index entries in Spanner.
2. **Prepare:** Backend sends the proposed change to the **Real-time Cache**.
3. **Commit:** Spanner commits the rows to Entities and IndexEntries atomically, using Spanner's **TrueTime** for external consistency.
4. **Accept:** The Real-time Cache records the change and prepares to notify listeners.



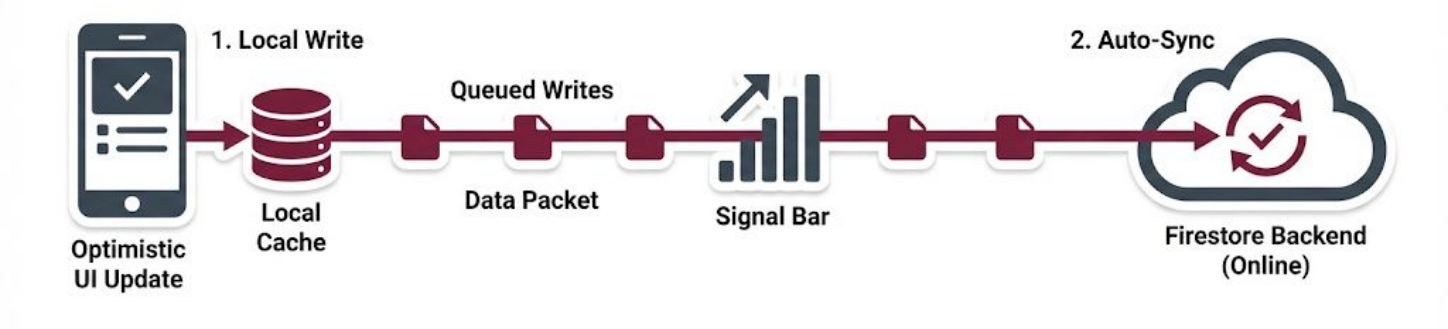
Architecture of ”The Listen”

- **The Problem:** How to notify millions of devices of changes efficiently?
- **The Solution:**
  1. **In-Memory Changelog:** Buffers recent changes in timestamp order.
  2. **Query Matcher:** Matches incoming document updates against active queries.
- **Mechanism:**
  - Clients receive an initial **Snapshot** (full data).
  - Subsequently, they receive only **Deltas** (diffs).
  - Guaranteed strict ordering via Spanner timestamps.



”Offline-First” Principles

- **Local Cache:** SDKs maintain a local database on the device.
- **Optimistic Concurrency:**
  - Local writes are applied immediately to the UI (zero latency).
  - Writes are queued and sent to the backend when connection is restored.
- **Conflict Resolution:** ”Last update wins” for blind writes; transactions re-validate upon reconnection.



Summary of Contributions

- **Simplified Development:** Combines schemaless flexibility with powerful indexing.
- **Serverless Scaling:** Multi-tenant architecture allows scaling from free-tier to enterprise without migration.
- **Strong Consistency:** Leverages Spanner to provide ACID guarantees in a NoSQL context.
- **Mobile-Ready:** Solves the hard problems of real-time sync and offline data access.

Thank you for the attention!